

STATE: ARCHEUS

Simulation -> Presentation Architecture Roadmap

Architecture Decision Record + Implementation Plan

Phase 11 - Facts, Timeline-backed action presentation, generic reactions,
versioned state projection, failure recovery, and live-service scalability

Prepared for the current State: Archeus Unity 6 / DOTS architecture

28 August 2026

Executive Decision

VERDICT - COMMIT TO THIS ARCHITECTURAL SHAPE

The safest high-end architecture is not “Unity Timeline everywhere” and it is not “a custom presentation VM everywhere.” Commit to a backend-agnostic presentation core: simulation emits immutable presentation-safe facts and versioned state projections; a presentation planner binds those facts to reusable recipes; a presentation scheduler executes the resulting plan through interchangeable backends. Use Unity Timeline/Playables as the primary authored-action backend now, and use generic lightweight presenters for common reactions. Reserve a Program/VM backend as an optional future executor. This keeps the hard contracts stable even if the playback technology changes.

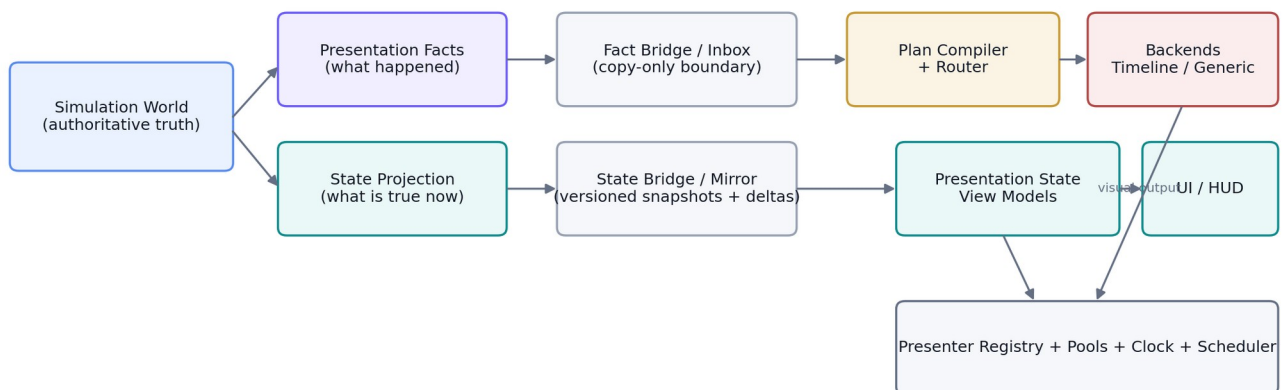
This is the architecture I would choose now for State: Archeus. The part worth “locking in” is the one-way data contract and the backend-neutral execution model, not a dependency on a single Unity presentation technology. That distinction is what makes the decision safe. If profiling later shows that raw Timeline is too heavy for a particular class of action, the project can add a lightweight Playables or custom program executor without changing the simulation fact format, the action grouping rules, the state bridge, the presenter registry, or the authoring semantics.

Why this is a stronger commitment than choosing Timeline versus a VM

The irreversible risk would be making PresentationFact, PresentationPlan, or gameplay content directly encode Timeline tracks, PlayableDirector objects, animation clip names, UnityEvents, or marker times. That would make Timeline the architecture. The recommended design instead makes Timeline one implementation behind a stable presentation backend contract. The same plan can theoretically be executed by a Timeline backend, a lightweight generic backend, an immediate fallback backend, or a future compact program backend.

- Simulation never waits for animation, VFX, UI, or a Timeline marker.
- Presentation never recalculates gameplay results.
- Cross-world data uses stable runtime IDs, never SimulationWorld Entity values or GameObject references.
- Facts describe committed observable outcomes; state projections describe current authoritative truth.
- Action presentation and generic fact reactions are separate: a five-second FTS may use Timeline, while a poison tick should use a lightweight generic presenter.
- Authored semantic slots such as Impact0 are stable contracts. A Timeline marker can implement the slot today; a future VM opcode can implement the same slot tomorrow.
- Missing or malformed presentation content can degrade visually, but must never deadlock or corrupt simulation.

Two one-way presentation bridges; one shared runtime shell



The stable contracts sit above the playback backend. This is the core anti-lock-in decision.

Confidence and the Honkai: Star Rail comparison

No public source exposes Honkai: Star Rail's exact combat-presentation implementation, so this document does not claim that HoYoverse uses Unity Timeline, a presentation VM, these class names, or these exact data structures. "HSR-like" is used here as a design target: a content-heavy turn-based game with many characters, cinematic ultimates, repeated generic reactions, speed controls, skins, VFX, audio, camera choreography, and a need for new content to be added mostly through tools rather than engine rewrites.

A production at that scale almost necessarily needs the same categories of separation even if the internal technology is proprietary: authoritative gameplay independent from visuals; reusable authored presentation assets; semantic binding of runtime results into those assets; generic handling of repeated common reactions; asynchronous/prefetched content; validation; pooling; failure recovery; and a presentation scheduler that can overlap some channels while serializing others. Unity's Playables and Timeline are designed for reusable scheduling, scene binding, animation/audio outputs, and custom markers, which makes them a sensible first backend for the current Unity 6 project rather than something the architecture must be permanently married to.

Roadmap at a Glance

Layer	Purpose	MVP implementation	Long-term scale path
Simulation facts	Describe what happened	PresentationFact buffer + central emitter	Typed fact contracts, action correlation, validation
Fact bridge	Move facts between worlds	Copy-only outbox/inbox	Double-buffer/ring-buffer, sequence auditing
Action assembler	Group facts into visual units	ActionInstanceID packets	Nested/related actions, look-ahead compilation
Presentation router	Choose how a fact/action should be shown	Timeline vs Generic vs NonVisual/Fallback	Policy tables, lane claims, content variants
Plan compiler	Bind runtime data to reusable content	Resolve recipe, slots, presenters	Burst-friendly data stage + main-thread Unity binding stage
Timeline backend	Play authored choreography	PlayableDirector + custom semantic markers	Pooled directors, custom tracks, variants, cinematic channels
Generic backend	Show common repeated facts cheaply	Damage/heal/effect/death presenters	Batched overlays, pooled VFX, specialized reaction graphs
Presentation scheduler	Control order and overlap	Primary + reaction + overlay lanes	Resource/lane claims, priorities, safe points, skip policies
State projection	Describe what is true now	Versioned view state	Snapshots + deltas, selective projections, reconciliation
Presenter registry	Resolve RuntimeID to visuals	Dictionary/registry of thin presenters	Compact handles, pooled presenters, skins/variants

Part I - Non-Negotiable Architectural Invariants

1. Simulation is authoritative; presentation is interpretive

Every numerical outcome, target choice, crit result, death, effect application, card movement, action-point cost, turn change, and battle-state transition is decided in SimulationWorld. PresentationWorld receives already-resolved truth and decides only how to communicate that truth. A marker at 0.43 seconds means "reveal the already-resolved impact now," never "deal damage now."

RULE

Animation timing may change the perceived timing of an outcome, but it must never change whether the outcome happened or what its gameplay result was.

2. The cross-world boundary is intentionally narrow

Simulation components, BlobAsset internals, behaviour indexes, VM stacks, RNG rolls, and Unity ECS Entity values are implementation details. They should not leak into presentation merely because they are convenient. The presentation contract should expose the smallest resolved semantic payload required to communicate the outcome. This keeps presentation decoupled from simulation refactors and prevents accidental access to hidden or future information.

3. Timeline is a backend, not a contract

ActionPresentationDefinition may select Timeline as its backend and reference a TimelineAsset, but PresentationFact and the generic PresentationExecutionPlan should not contain Timeline track objects. Semantic roles and slot IDs are the contract: SourceAnimator, PrimaryTarget, Impact0, Impact1, Aftermath, Finish. The Timeline backend maps those stable concepts onto Timeline tracks and markers. A future Program backend can map exactly the same concepts onto opcodes without touching gameplay or facts.

4. Persistent state and historical facts remain separate

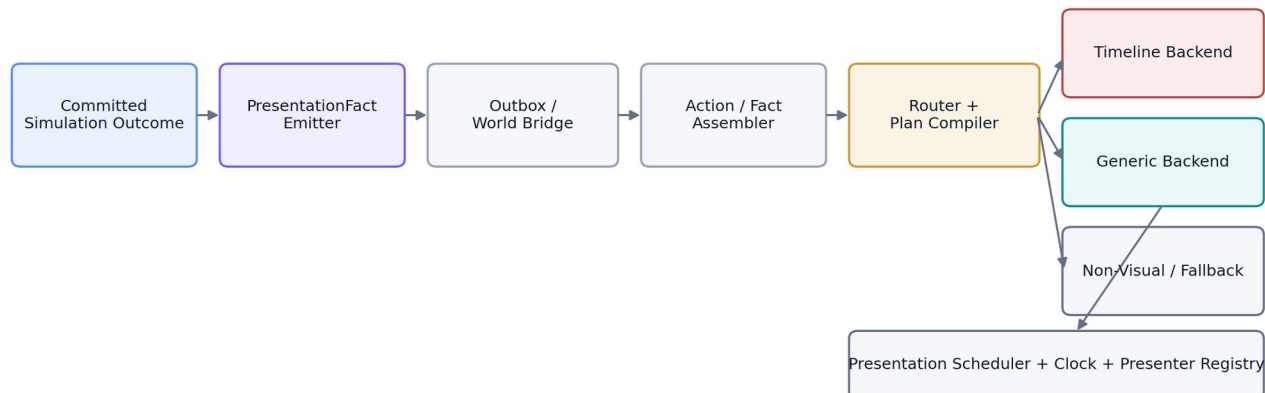
Facts answer “what happened?” State projections answer “what is true now?” The presentation layer needs both. Facts provide temporal causality and animation ordering; state projections initialize and reconcile the visible world. Health illustrates the distinction: authoritative state may already be 50,000 HP while presentation is still animating two queued hits from 100,000 -> 80,000 -> 50,000. When the timeline is caught up, presented state must converge to authoritative state.

5. Failure must degrade presentation, never simulation

A missing presenter, animation clip, marker, VFX, TimelineAsset, or malformed slot binding is a presentation defect. It should be logged, replaced with a generic fallback if possible, and consumed so the battle can continue. A missing marker must never create an infinite wait. A failed Timeline must never leave required damage facts unconsumed forever.

Part II - Simulation -> Fact -> Presentation

Simulation -> Fact -> Presentation



Fact presentation is event/history-oriented: immutable outcomes move forward, are classified, bound, and played.

6. PresentationFact: the public semantic event contract

A **PresentationFact** is an immutable, presentation-safe statement about a committed simulation outcome. It is not a command to play a specific asset. It exists because the internal **BattleEvent** pipeline is far too detailed and unstable to expose directly. **DamageRequested**, **DamageCalculated**, **DamageMitigated**, and resolver-specific stages can change without affecting the presentation API. **DamageApplied** is the stable observable truth.

```

public struct PresentationFact : IBufferElementData
{
    public ulong Sequence;
    public uint BattleRuntimeID;

    public PresentationFactType Type;

    // Gameplay/presentation correlation, not scheduler ownership.
    public uint ActionInstanceID; // 0 when no authored action owns the fact
    public uint ActionDefinitionID; // stable content identity / presentation lookup key

    // Causal metadata that may help presentation group or diagnose results.
    public uint GroupID;
    public ushort Generation;

    // Cross-world stable identities. Never Unity.Entities.Entity.
    public uint SourceRuntimeID;
    public uint TargetRuntimeID;

    // Semantic sub-action identity: one AOE hit can have many targets,
    // while a five-hit attack uses five different hit indexes.
    public ushort HitIndex;

    public PresentationFactPayload Payload;
}
  
```

Recommended fact categories

Category	Examples	Why it exists
Action lifecycle	ActionDeclared / ActionStarted / ActionCompleted	Provides an authored visual unit and a seal point for packet assembly.
Combat outcomes	DamageApplied / HealApplied /	Resolved numerical changes the player

Category	Examples	Why it exists
	ShieldChanged	can observe.
Effect outcomes	EffectApplied / EffectRemoved / EffectExpired	Status visuals and UI can react without knowing effect pipeline internals.
Entity lifecycle	EntityKilled / Revived / Spawned / Removed	Allows death, revive and spawn presentation to be scheduled truthfully.
Turn/battle outcomes	TurnStarted / TurnEnded / Victory / Defeat	Can route to global UI or larger sequences.
Card/action UI outcomes	CardPlayed / CardDrawn / ResourceSpent	Useful where the visual change is better represented as a historical transition than a raw state poll.

What a fact must not contain

- SimulationWorld Entity values or pointers to ECS components.
- Animator, GameObject, Transform, TimelineAsset, AudioClip, VFX prefab, or camera references.
- Intermediate damage formula values unless a real visual feature needs them.
- RNG rolls, hidden enemy intent, unrevealed cards, secret behaviour memory, or future simulation outcomes.
- Marker times such as 0.43 seconds. HitIndex is semantic; marker time is presentation content.

7. Central PresentationFactEmitter

The fact emitter is the semantic firewall. Resolver or action code should not manually fill arbitrary fields. Typed helper methods make it difficult to leak bad data and make auditing straightforward. The emitter also assigns Sequence and copies only permitted causal/action metadata.

```
PresentationFactEmitter.EmitDamageApplied(
    ref factBuffer,
    in presentationContext,
    sourceRuntimeID,
    targetRuntimeID,
    finalDamage,
    didCrit,
    hitIndex);
```

This is analogous to the central BattleEventEmitter now used for structural event metadata. The difference is purpose: BattleEventEmitter protects simulation causality; PresentationFactEmitter protects the public visual contract.

8. Sequence, GroupID, Generation, and ActionInstanceID are different axes

Do not overload one identifier to do four jobs. Sequence is an absolute presentation-safe ordering number. GroupID describes the root causal chain. Generation describes causal wave. ActionInstanceID describes a local authored action that should normally receive one coherent visual performance. These values can overlap in non-trivial ways: one GroupID can contain an original attack, a counterattack, and an additional attack, each with separate ActionInstanceIDs.

Field	Meaning	Presentation use
Sequence	Canonical order in the emitted fact stream	Exactly-once transfer, diagnostics, deterministic fallback ordering.
GroupID	Ultimate root causal action	Group diagnostics, optional higher-level batching.
Generation	Causal wave	Useful context for reaction ordering, never a replacement for ActionInstanceID.
ActionInstanceID	One local gameplay action/performance	Primary key for action packet assembly and bespoke animation selection.
HitIndex	Semantic hit within an action	Maps multi-hit outcomes to Impact0/1/2...

Field	Meaning	Presentation use
		without exposing animation time.

9. Multi-target and multi-hit semantics

The fact model should make the two most common cases unambiguous. One AOE hit against five targets produces five DamageApplied facts with the same ActionInstanceID and the same HitIndex, but different TargetRuntimeIDs. Five rapid hits against one target produce five DamageApplied facts with the same ActionInstanceID but HitIndex 0..4. This makes presentation binding simple and scalable.

```
AOE single impact:
  Action 581 / Hit 0 / Target A
  Action 581 / Hit 0 / Target B
  Action 581 / Hit 0 / Target C

Five-hit finisher:
  Action 582 / Hit 0 / Target A
  Action 582 / Hit 1 / Target A
  Action 582 / Hit 2 / Target A
  Action 582 / Hit 3 / Target A
  Action 582 / Hit 4 / Target A
```

An authored Timeline may therefore have one Impact0 marker that consumes all HitIndex=0 facts for the AOE, or five markers Impact0..Impact4 placed very close together for the finisher. Later tooling can add a burst/range marker that expands a cadence automatically, but the underlying fact contract does not change.

10. Fact outbox and cross-world bridge

The SimulationWorld should own a per-battle PresentationFact outbox. A bridge system that runs after the authoritative simulation update copies plain-data facts into a root-level cross-world transport. PresentationWorld pulls them into its own inbox before presentation planning. The bridge copies values; it never shares DynamicBuffer handles, Entity values, Blob references, or UnityEngine.Object references across worlds.

WHY COPY INSTEAD OF SHARE

World-local ECS handles are not stable presentation identities, and direct cross-world reads create hidden lifetime/order dependencies. A tiny copy cost buys a clean ownership boundary, replayability, easier diagnostics, and future network/replay compatibility.

Recommended transport evolution

- MVP: double-buffered NativeList/managed struct queue owned by a root PresentationBridge service.
- Next: per-battle sequence tracking and acknowledgements so exactly-once consumption is auditable.
- Scale: ring buffers or chunked pages to avoid reallocations during long battles.
- Overflow: debug builds fail loudly; release builds may force presentation fast-forward/reconciliation rather than blocking simulation.

11. Action/fact assembler

Presentation does not want to start the Karathos normal attack three times because three targets took damage. The assembler groups related facts by ActionInstanceID into an immutable ActionPacket. ActionStarted/ActionCompleted, or equivalent action-envelope metadata, provides a natural seal point. Non-action facts such as a DOT tick may bypass action assembly and route directly as generic facts.

```
ActionPacket #581
  ActionDefinitionID: Karathos_Normal
  SourceRuntimeID: 103
  Facts:
    DamageApplied Hit0 -> Target 201
    DamageApplied Hit0 -> Target 202
    DamageApplied Hit0 -> Target 203
    EntityKilled      -> Target 203
```

Sealed: true

Keeping the packet separate from the eventual backend is important. A packet can be routed to a bespoke Timeline, a generic attack recipe, an immediate fallback, or a future compact program without changing its structure.

12. Presentation router: classify before executing

The router answers “what class of presentation should communicate this truth?” It does not live in simulation because whether something deserves a five-second cinematic is a visual/content decision. The router should consider action definition, fact context, content availability, user settings, accessibility/skip policy, and fallback rules.

Route	Typical content	Execution style
Authored action	Normal, Supercharged, FTS, counter, backup attack	Timeline backend in the MVP; reusable choreography with semantic slots.
Generic reaction	DOT tick, heal tick, common shield flash, basic status application	Lightweight pooled presenter; no Timeline needed.
Overlay	Damage/heal numbers, crit indicator, small text	Independent high-throughput overlay lane.
Global/UI	Turn banner, victory banner, simple field notification	UI presenter or optional short authored recipe.
NonVisual	Debug/correlation facts with no player-facing meaning	Mark intentionally consumed with no visual output.
Fallback	Missing recipe/presenter/asset/slot contract	Generic truthful representation; never deadlock.

Part III - Presentation Plan Compiler -> Execution Plan -> Runtime Backends

13. The most important anti-lock-in layer: backend-neutral PresentationExecutionPlan

This is the architectural refinement that makes the Timeline decision safe. The plan compiler should not output “a Timeline.” It outputs a backend-neutral execution plan containing semantic bindings and a chosen recipe/backend. Timeline is one executor capable of consuming that plan. If a future profile shows that a particular class of content needs a custom VM or direct PlayableGraph, a second executor can consume the same plan contract.

```
public enum PresentationBackendKind : byte
{
    Timeline,
    Generic,
    ImmediateFallback,
    // Reserved future option:
    Program
}

public struct PresentationExecutionPlan
{
    public uint PlanID;
    public uint ActionInstanceID;
    public uint RecipeID;
    public PresentationBackendKind Backend;
    public PresentationLaneMask LaneClaims;
    public PlaybackPolicy PlaybackPolicy;

    // Compact ranges into pooled binding arrays.
```

```
public int PresenterBindingStart;
public int PresenterBindingCount;
public int FactSlotBindingStart;
public int FactSlotBindingCount;
}
```

What the compiler actually does

- 1. Resolve the action/fact packet to a PresentationRecipe using stable content IDs and fallback policy.
- 2. Validate that the recipe can represent the packet: required semantic slots exist and facts have legal mappings.
- 3. Resolve RuntimeIDs to thin presenter handles through PresentationRegistry.
- 4. Bind semantic fact groups to slots such as Impact0, Impact1, Aftermath, Finish.
- 5. Resolve the recipe’s resource/lane claims: PrimaryAction, Camera, Reaction, Overlay, Environment, etc.
- 6. Resolve/preflight Unity-side assets required by the chosen backend.
- 7. Produce a compact plan. Playback should not need to rediscover targets, search strings, or inspect simulation data.

Two-stage compilation for future scale

If profiling ever justifies it, compilation can be split. Stage A is pure-data classification and slot binding, suitable for Jobs/Burst or background work. Stage B is a short main-thread binding pass that touches UnityEngine.Object references such as TimelineAsset, Animator, PlayableDirector, Camera, or VFX pools. The MVP can do both on the main thread because turn-based action frequency is low; the data model should simply avoid making a future split impossible.

14. Why Timeline/Playables should be the first authored-action backend

Unity Timeline already provides the expensive authoring experience a custom presentation VM would eventually need to recreate: a visual time axis, clip placement and duration, scrubbing, animation/audio sequencing, track binding, custom tracks, marker authoring, and integration with the Playables graph. PlayableDirector instantiates and controls a PlayableGraph from a PlayableAsset, and Unity exposes runtime generic binding so the same TimelineAsset can be bound to whichever runtime character instance is actually acting.

The project therefore gains strong visual tooling immediately without making Timeline the semantic source of truth. Archeus owns action packets, semantic slots, runtime IDs, fact binding, scheduling, speed policy, fallback, and validation. Timeline owns authored temporal choreography for content that benefits from a visual sequencer.

Why not a custom presentation VM as the only backend today?

Custom VM advantage	Why it is not enough to justify replacing Timeline now
Compact deterministic runtime instructions	Presentation is not the simulation determinism bottleneck; visual asset evaluation dominates.
Full control over scheduling semantics	Archeus already retains scheduling control above the backend.
Potentially cheaper for tiny sequences	Tiny sequences should use Generic/Immediate backends rather than force every fact through Timeline.
No dependency on Timeline package behavior	A backend abstraction already provides an escape hatch if Timeline proves unsuitable.
Easy code-generated sequences	A future Program backend can be added later without changing facts, recipes, slots, or state projection.

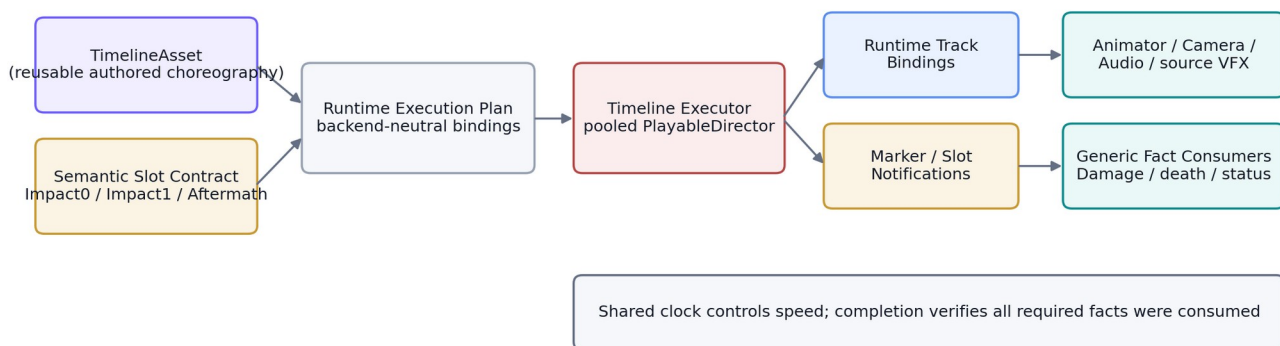
Why not raw Timeline as the entire presentation architecture?

Raw Timeline failure mode	Archeus wrapper that fixes it
Direct scene/GameObject bindings make assets instance-specific	Bind standardized roles dynamically at runtime through the recipe/registry.
UnityEvents and strings become combat semantics	Use typed custom semantic markers/slots and stable IDs.
Dynamic AOE targets create combinatorial target tracks	Timeline controls known roles; slot consumption fans out to

Raw Timeline failure mode	Archeus wrapper that fixes it
	generic target presenters.
Every DOT/status tick would require an authored asset	Route common facts to Generic backend.
Missing marker can hang bespoke callback logic	Runner has watchdog/fallback consumption and end-of-plan reconciliation.
Timeline becomes hard to replace	ExecutionPlan is backend-neutral and Timeline-specific data stays inside TimelineRecipe/Executor.

15. Timeline-backed recipe and semantic slots

Timeline is a backend, not the architecture



The TimelineAsset is reusable content. Runtime facts live in the plan, not inside the asset.

An ActionPresentationDefinition (or PresentationRecipe) references a TimelineAsset plus a stable semantic contract. The contract says which runtime roles must be bound and which fact slots the authored sequence can consume. It does not contain target-specific runtime values.

```

Karathos_Normal_Presentation
Backend: Timeline
Timeline: Karathos_Normal.timeline

Required role bindings:
SourceAnimator
BattleCamera
BattlefieldRoot

Semantic slots:
Impact0    -> DamageApplied(HitIndex = 0), ConsumeAll, optional if dodged
Aftermath  -> EntityKilled / EffectApplied, ConsumeAll
Finish     -> completion safe point

Fallback recipe:
Default_Normal_Attack

```

Custom Timeline marker

Create a typed marker such as PresentationConsumeMarker that implements the Timeline marker/notification interfaces. It stores compact semantic data: SlotID, optional HitIndex/range, consumption mode, and whether the slot is required. The marker does not contain damage values, runtime targets, or simulation references. On notification, the Timeline executor asks the current RuntimePresentationPlan to consume the bound slot.

```
[Serializable]
public sealed class PresentationConsumeMarker : Marker, INotification
{
    public PresentationSlotID Slot;
    public FactConsumptionMode Consumption;

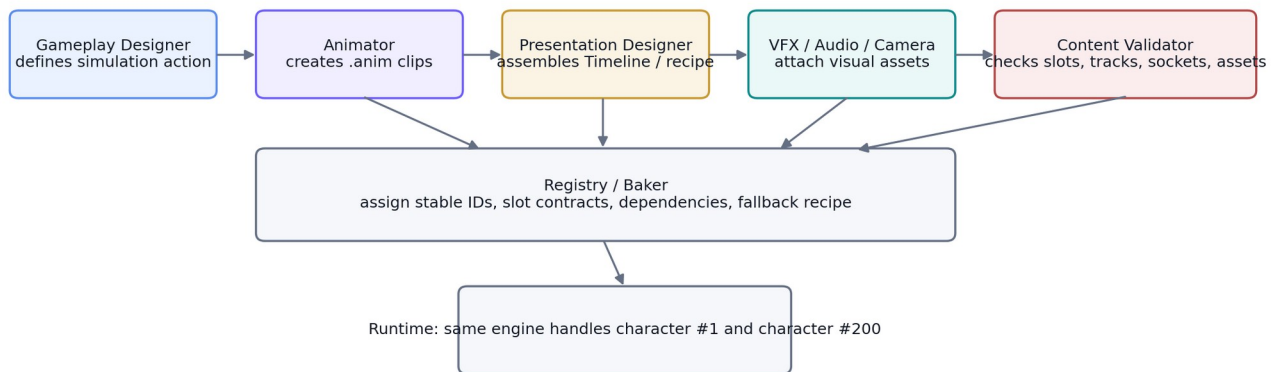
    public PropertyName id => new PropertyName("ArcheusPresentationConsume");
}
```

Track binding roles

Do not discover tracks by display-name strings during every attack. A content validator/baker should identify which output track corresponds to SourceAnimator, Camera, Battlefield, or another standardized role. At runtime the Timeline executor uses PlayableDirector.SetGenericBinding to bind those output tracks to already-resolved presenter components. Dynamic targets normally do not receive dedicated Timeline tracks.

16. Karathos authoring workflow from raw .anim clips

Future content becomes mostly authoring, not engine code



The long-term goal: adding character #200 is predominantly content assembly and validation.

8. Animator delivers karathos_normal.anim, karathos_supercharged.anim, and karathos_fts.anim. These clips contain motion only; they do not deal damage or know simulation state.
9. A thin KarathosPresenter prefab is created or verified. It exposes an Animator and standardized sockets such as Root, HitCenter, Weapon, Head, and VFXOrigin. It contains no combat rules.
10. A presentation/technical animator creates Karathos_Normal.timeline. The normal animation clip is placed on a source Animation Track. Camera, source VFX, and fixed audio can be authored visually on their own tracks.
11. At the contact frame, the author places a PresentationConsumeMarker for Impact0. For a five-hit finisher, markers Impact0..Impact4 may be placed close together, or future tooling may author a single Burst marker with count/cadence.
12. The presentation definition declares the TimelineAsset, required role bindings, slot contract, lane claims, fallback recipe, and asset dependency bundle/label.
13. The content validator checks that required markers exist, required binding roles are present, referenced assets exist, presenter sockets expected by dynamic VFX are supported, and fallback content is valid.
14. The registry/baker assigns stable IDs and builds compact runtime lookup data. No gameplay code changes are required to add the finished animation set.
15. At runtime, simulation emits action/fact data; the compiler resolves Karathos_Normal_Presentation, binds the actual runtime Karathos presenter and current target facts, then the Timeline executor plays the reusable asset.

17. Dynamic targets: keep them out of static Timeline tracks

The authored Timeline should directly control roles whose identity is known structurally: the source character, battle camera, battlefield/environment, and fixed source-side VFX. Dynamic targets should normally be revealed through semantic fact slots. This avoids authoring “Target 1, Target 2, Target 3...” tracks and scales from one target to dozens without changing the TimelineAsset.

At Impact0, the slot may contain three DamageApplied facts. A generic DamageFactPresenter fans those facts out to three already-resolved target presenters, triggering hit reactions, damage numbers, visual HP transitions, and target-side VFX. The attack Timeline remains one reusable asset.

18. Multi-hit presentation is easy and remains semantic

If the end of an attack deals five sequential damage instances, the Timeline may indeed contain five impact markers placed extremely close together. Each marker consumes a different HitIndex. This is not a performance concern by itself; five marker notifications are tiny compared with animation/VFX work. The important rule is that simulation emits five semantic hits, not five timestamps.

For authoring convenience, later custom tooling can add a BurstConsume marker with FirstHit, Count, and Cadence. The marker can schedule internal consumption callbacks against the presentation clock. That is an editor/productivity enhancement, not an architectural change.

19. Generic presentation backend: the scalability workhorse

Most repeated visual reactions should not require Timeline. A poison DOT tick, heal tick, shield pulse, generic effect application, damage number, common death, or small UI notice should route through lightweight generic presenters. These presenters are pooled, data-driven, and can run concurrently with an authored Timeline.

Fact / action	Likely backend	Reason
Karathos FTS	Timeline	Cinematic camera/VFX/audio choreography benefits from visual sequencing.
Karathos normal attack	Timeline or generic action recipe	Character-specific motion and impact timing may justify authored choreography.
DOT damage tick	Generic	No need to instantiate/play a cinematic sequence for one number + small VFX.
Damage number at Impact0	Generic overlay consumer	The Timeline only reveals the fact at the right time.
Common hit reaction	Generic reaction presenter	Dynamic target count; reused across hundreds of abilities.
Turn banner	Generic UI or small recipe	Independent from character animation.

Part IV - Presentation Scheduler, Timing, Speed, and Skipping

20. Presentation needs its own scheduler - but it is not the simulation scheduler

Simulation scheduling decides causality. Presentation scheduling decides communication: what may overlap, what claims the camera, what must wait for a primary action, and what may continue independently. It should never change outcomes. A small lane/resource model is sufficient and scales far better than a single global queue.

Suggested lane model

PrimaryAction	- normal / supercharged / FTS / counters
Reaction	- target hit reaction / death / revive
Overlay	- damage numbers / heal numbers / small text
Camera	- exclusive camera choreography when claimed
Environment	- battlefield-wide authored effects
UI	- turn banners / combat notices
Ambient	- persistent low-priority effects

A Timeline-backed FTS may claim PrimaryAction + Camera + Environment. Damage-number overlays may still run when its impact marker consumes facts. A DOT tick may wait for a presentation safe point or run in the Overlay/Reaction lanes depending on the game's visual policy. These are content/presentation rules rather than changes to simulation ordering.

21. PresentationClock and variable speed

Use a single authoritative presentation clock/policy that all visual subsystems consult. Timeline playback speed can be changed through the Playables graph; generic tweens, hit reactions, VFX time scale, camera transitions, and UI animation durations should derive from the same presentation speed policy. Audio may deliberately preserve pitch or apply a separate policy.

```
PresentationClock
  VisualSpeed: 1.0 / 2.0 / 3.0
  SkipPolicy: None / Fast / SkipNonEssential / Immediate
  AudioPolicy: PreservePitch / SelectiveAcceleration
```

Do not bake "2x versions" of Timelines. A Timeline marker remains at authored local time 0.43s; changing the Playable speed changes when that local time is reached in real time. The same action/facts remain valid.

22. Skip and fast-forward are first-class policies

Because simulation has already resolved, skipping is primarily a presentation operation. The runner can stop non-essential choreography, consume all required fact slots in canonical order, push visual state toward authoritative state, and release backend resources. A long FTS can therefore be skipped without asking simulation to re-run or alter the action.

- Skip must still consume required facts so HP, death, effect visuals, and UI do not get stranded.
- Generic overlays may be compressed or suppressed according to accessibility/performance settings.
- The runner should expose a completion reason: PlayedNormally, FastForwarded, Skipped, Fallback, or FailedRecovered.

23. Plan completion invariant

SHIPPING INVARIANT

When a plan finishes, every required fact assigned to that plan is either Consumed, ExplicitlyNonVisual, or FallbackConsumed. "Unaccounted for" is a bug. This one invariant catches missing markers, malformed contracts, target-resolution failures, and content mistakes early.

Part V - Failure Recovery and Content Validation

24. Compile-time / build-time validation first

The cheapest presentation bug is the one caught before entering Play mode. A validator should eventually scan all PresentationDefinitions and their variants/skins. It can verify required TimelineAsset references, binding-role tracks, semantic markers, marker uniqueness/range rules, fallback recipes, presenter socket contracts, and asset dependency labels. This is exactly the kind of tooling that becomes disproportionately valuable when content count grows.

25. Runtime mismatch policies

Mismatch	Required behavior
Missing TimelineAsset	Resolve fallback recipe; if absent, use Generic/Immediate truthful presentation.
Missing source presenter	Do not start a broken authored sequence. Consume through fallback and log RuntimeID/recipe/action.
Missing target presenter	Skip target-specific visual, still consume the fact; reconcile state later.
Missing required marker	Compiler should reject; if discovered at runtime, watchdog/end-of-Timeline fallback consumes remaining slot.
Timeline ends with unconsumed facts	Log structured error; fallback-consume in Sequence order before plan completion.
Marker asks for fact that does not exist	If optional, no-op. If required, record contract violation and continue with fallback policy.
Extra HitIndex with no slot	Compiler maps to generic Aftermath/fallback slot or immediate generic presentation; never silently discard.
Asset load failure	Fallback recipe or simplified generic visuals; never stall simulation waiting indefinitely.

26. Watchdogs and deadlock prevention

Any blocking presentation operation must have a bounded failure path. A Timeline should normally complete naturally; however, if a custom wait/signal is introduced later, it requires an expected maximum duration and a recovery path. If an Impact marker does not fire because the content is malformed, plan finalization should reveal remaining required facts rather than wait forever. “Ugly but truthful” is always preferable to a soft-lock.

27. Diagnostics should preserve the semantic chain

Structured logs should include BattleRuntimeID, Sequence range, ActionInstanceID, ActionDefinitionID, RecipeID, backend, GroupID/Generation where relevant, SlotID, SourceRuntimeID/TargetRuntimeID, and completion reason. This makes a bug report such as “skin 4 FTS sometimes loses the third hit” traceable without dumping simulation internals into presentation.

Part VI - Performance and Live-Service Scale

28. Where presentation performance actually goes

The plan compiler and marker dispatch should be made cheap, but they are unlikely to dominate frame cost. In a polished Unity combat scene, the heavier costs are more likely to be skeletal animation evaluation, rendering/skinning, VFX/particles, transparent overdraw, post-processing, camera effects, UI rebuilds, asset loading, and allocation/GC spikes. The architecture should therefore avoid adding avoidable overhead in the hot path while keeping optimization effort focused on the real visual workload.

Runtime hot-path rules

- Resolve presenters and content references before playback. No GameObject.Find, repeated GetComponent chains, or string-based track lookup at impact time.
- Pool PlayableDirectors or keep a small reusable set by lane rather than creating/destroying one per attack.
- Pool damage-number widgets, projectiles, transient VFX, hit decals, and common reaction objects.
- Prefer stable integer/hash IDs for slots, sockets, recipes, and animation roles. Human-readable strings belong in authoring/debugging, not repeated runtime search.
- Reuse binding arrays/lists. Avoid per-fact managed allocation when a pooled/native structure will do.

- Compile a small look-ahead horizon instead of creating heavy execution objects for the entire unresolved future battle presentation.
- Keep SimulationWorld free to continue. Presentation backlog is handled with queues, speed-up, or fallback - not backpressure into gameplay.

29. Asset preloading and residency

For the MVP, keeping presentation assets resident is acceptable. A live-service-scale content pipeline should eventually derive battle presentation dependencies from the participating characters, skins, environment, and expected common effects, then preload them before or during battle setup. Unity Addressables supports asynchronous loading of assets and dependencies, which is a natural later integration point. The presentation recipe should refer to stable content IDs/handles so the asset-loading strategy can evolve without changing fact contracts.

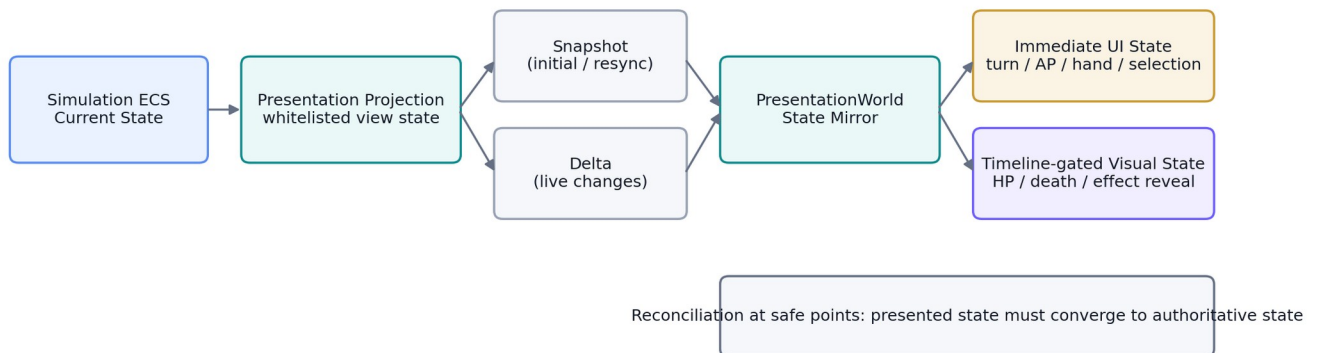
30. Why this scales to hundreds of characters

The marginal cost of a new character should be mostly content: animation clips, Timeline recipes for distinctive actions, VFX/audio/camera assets, and validation. Generic damage/heal/death/status presenters, the fact contract, planner, scheduler, clock, bridge, and state projection are shared. A character should not require new engine code merely because their normal attack has a different number of targets or impact markers.

Special mechanics may still require new semantic capabilities - for example a novel visual role or a new fact type - but that is deliberate engine extension rather than bespoke per-character scripting. The architecture makes the common path cheap and the rare path explicit.

Part VII - Simulation -> State -> Presentation

Simulation -> State -> Presentation



State presentation is projection/reconciliation-oriented, not historical playback.

31. Why a second pipeline is necessary

Facts alone cannot efficiently answer “what does the battle look like right now?” A UI opened halfway through a battle should not replay every damage, draw, resource change, and effect event since turn one to reconstruct current state. Simulation therefore publishes a separate presentation-safe projection of current truth. This projection is still one-way and whitelisted; presentation never queries arbitrary SimulationWorld components.

32. Presentation state projection

A projection is a deliberately designed view model, not a memory copy of ECS. It should expose only the state presentation actually needs. The exact shapes can evolve independently from simulation component layouts.

```
public struct BattlePresentationState
{
    public uint BattleRuntimeID;
    public ulong Revision;
    public int TurnNumber;
    public BattlePhaseView Phase;
    public int ActionPoints;
    public uint ActiveLeadRuntimeID;
}

public struct CharacterPresentationState
{
    public uint RuntimeID;
    public uint DefinitionID;
    public BattleSide Side;
    public float CurrentHealth;
    public float MaxHealth;
    public bool IsAlive;
    public int FieldSlot;
}
```

33. Snapshot + delta is the long-term shape

Use full snapshots for initialization, scene re-entry, presentation reset, reconnect/resync, and debugging. Use deltas for normal live updates so presentation does not copy every character, card, effect, and UI state every frame. Each publication carries a monotonically increasing Revision so PresentationWorld can detect missed updates and request/accept a fresh snapshot.

Mechanism	Use	Properties
Snapshot	Battle start, presenter rebuild, mismatch recovery	Self-contained authoritative projection at revision N.
Delta	Normal live play	Small change set: HP changed, card entered hand, AP changed, turn advanced, etc.
Revision	Every state publication	Detect gaps/out-of-order application and make reconciliation deterministic.

34. PresentationWorld maintains a mirror

PresentationWorld owns its own entities/view-state storage keyed by stable RuntimeID. The mirror is designed for visual consumption: CharacterPresentationState, HandPresentationState, EffectPresentationState, BattleHUDState, and similar projections. Thin GameObject presenters or UI view models subscribe to/observe this local mirror rather than reaching back into SimulationWorld every frame.

35. Immediate state versus timeline-gated visual state

Not every projected value should be shown immediately. Some values are safe to reflect as soon as the state mirror updates; others should be visually revealed by facts so the UI does not “spoiler” an animation. This is a presentation policy, not a contradiction between the two pipelines.

State	Typical policy	Reason
Turn number / phase	Usually immediate or banner-gated	Current UI context; little value in lagging far behind.
Action points	Often immediate after command acceptance	Communicates resource commitment promptly; can still animate the decrement.

State	Typical policy	Reason
Hand/deck membership	Depends on card animation	State is authoritative; draw/play facts can time the visible transition.
Current HP	Timeline-gated visual transition	Avoid jumping to final HP before queued hits visibly land.
Alive/dead	Fact-gated visual transition	Death should reveal at the correct impact/aftermath point.
Effects/buffs	Often fact-gated icon/VFX, state-backed reconciliation	Preserves action sequencing while guaranteeing eventual correctness.

36. The convergence invariant

STATE RECONCILIATION RULE

When the relevant presentation backlog is empty or a safe reconciliation point is reached, the visible/presented state must converge to the latest authoritative PresentationState revision. Facts explain the path; state guarantees the destination.

This makes recovery robust. If a damage-number VFX failed or a marker was missed, the next safe reconciliation can still correct the visible HP bar, alive/dead state, card layout, and effect icons. The state pipeline is therefore not redundant with facts; it is the correctness backstop for presentation.

37. State bridge mechanics

The state bridge should use the same world-ownership philosophy as facts: simulation builds a whitelisted projection, a root bridge copies it, and PresentationWorld applies it locally. For MVP, a simple snapshot or coarse delta buffer is enough. Later, use revisioned deltas and occasional snapshots. Avoid having MonoBehaviours or PresentationWorld systems query SimulationWorld components directly.

38. UI/view binding

Persistent UI should bind to PresentationWorld state or dedicated view models, not to BattleEvent history. Hand slots, turn counters, action points, selected target highlights, deck counts, and persistent effect icons are state views. Historical animations such as “card draws from deck into hand” may be triggered by facts, but the final hand membership comes from state.

39. Input remains a separate command path

The presentation state bridge is one-way. Player clicks, card choices, target selection, and skip/speed controls should not mutate the state mirror as if it were authoritative gameplay. Gameplay input travels through a separate command/request path into SimulationWorld, which validates and applies it. Presentation may optimistically animate some UI, but simulation remains the source of truth and can reject/repair invalid input.

40. State mismatch recovery

If PresentationWorld detects a revision gap, missing presenter, impossible hand layout, or divergent visible HP at a reconciliation point, the recovery path is to apply/request the latest complete projection snapshot and rebuild/repair local view state. It is not to inspect raw simulation components or replay an unbounded event history. This is the state equivalent of generic fallback consumption in the fact pipeline.

Part VIII - Shared Presentation Infrastructure

41. Stable runtime identity

RuntimeID is the universal cross-world join key. Simulation entities and PresentationWorld entities are different objects. GameObject presenters are also different objects. Stable IDs let facts, state projections, action packets, and presenter registries agree on identity without sharing world-local handles.

42. PresentationRegistry

PresentationRegistry maps RuntimeID to thin presenter handles. A CharacterPresenter exposes visual capabilities and standardized sockets; it does not contain gameplay rules. The registry also supports delayed creation: if an entity is spawned in simulation before its presentation object is ready, facts can queue/fallback while state later reconciles.

43. Recipe/content registry

A stable PresentationRecipeID maps gameplay presentation keys to backend-specific content. Variants such as skins can override the recipe while preserving the same semantic contract. A summer Karathos normal animation and base Karathos normal animation can both expose Impact0 and Finish even though their assets, timing, camera, and VFX differ.

44. Shared pools

Transient visual objects should be pooled centrally or by category. Damage numbers, generic hit effects, projectiles, small status VFX, and PlayableDirector instances are classic candidates. Pooling policy should not leak into fact or state contracts; it is an implementation detail of the presenter/backends.

45. Presentation variants and skins

Because simulation outputs stable semantic facts, skins are almost entirely presentation-side substitutions. Recipe resolution can consider CharacterDefinitionID + SkinID + action key and choose a different Timeline/VFX/audio package with the same required slot contract. This prevents cosmetic content from branching simulation logic.

Part IX - Detailed Implementation Roadmap

Milestone 0 - Freeze the contracts before coding the bridge

- Write the architecture decision record into the project: Timeline is a backend, semantic slots are the contract, simulation never waits for presentation.
- Choose stable types for PresentationFact, Sequence, RuntimeID, ActionInstanceID, HitIndex, PresentationRecipeID, PresentationSlotID, and state Revision.
- Define which current simulation events are externally observable enough to emit facts. Start small: ActionStarted/Completed, DamageApplied, EffectApplied/Removed, EntityKilled, TurnStarted/Ended.
- Do not implement rich character semantics, sophisticated scheduler lanes, or full asset streaming yet.

Milestone 1 - Fact production in SimulationWorld

- Add DynamicBuffer<PresentationFact> to battle root.
- Add PresentationFactSequenceCounter per battle or an equivalent battle-local monotonic sequence.
- Implement central PresentationFactEmitter.
- Emit DamageApplied after HP mutation is committed; emit EntityKilled after death is committed; emit action lifecycle facts around a minimal TestAttack action.
- Test that two identical deterministic simulations emit the same fact sequence and payloads.

Milestone 2 - Cross-world fact bridge

- Create root PresentationBridge service with two owned buffers: simulation write/export side and presentation read/import side.
- SimulationBridgeSystem copies completed facts after BattleSimulationGroup. PresentationBridgeSystem ingests before PresentationWorld planning.
- Log BattleRuntimeID + Sequence on both sides to prove no loss, duplication, or world-local Entity leakage.

Milestone 3 - Runtime identity and presenter registry

- Spawn a minimal CharacterPresenter GameObject for each on-field participant and register by CardRuntimeID/character runtime ID.
- Prove PresentationWorld can resolve SourceRuntimeID and TargetRuntimeID without accessing SimulationWorld.
- Add clean unregister lifecycle and missing-presenter diagnostics.

Milestone 4 - Generic Damage presenter vertical slice

- Route DamageApplied without an authored action to GenericDamagePresenter.
- For the first slice, log or show a tiny damage-number UI. Then add pooled damage numbers and a simple hit reaction.
- This proves facts can create visible output before Timeline exists, and establishes the fallback path used by DOTs forever.

Milestone 5 - ActionInstanceID and ActionPacket assembly

- Introduce the minimal action identity required for TestAttack and future normal/supercharged/FTS actions.
- Assembler groups facts by ActionInstanceID and seals packets at ActionCompleted.
- Add HitIndex and test one AOE single-hit packet versus one five-hit packet.

Milestone 6 - Backend-neutral PresentationExecutionPlan

- Implement PresentationRecipe registry and PresentationBackendKind.
- Plan compiler binds action packet facts to semantic slot IDs and resolves presenter handles.
- Initially support Timeline, Generic, and ImmediateFallback backends; keep Program reserved, not implemented.
- Add "all required facts accounted for" validation before and after execution.

Milestone 7 - Timeline backend MVP

- Add one pooled PlayableDirector owned by BattlePresentationRunner.
- Create custom PresentationConsumeMarker/notification receiver.
- Create TestAttack.timeline with Source Animation track and Impact0 marker.
- Bind the source Animator dynamically using standardized track-role metadata and SetGenericBinding.
- At Impact0, consume bound DamageApplied facts through GenericDamagePresenter.
- On Timeline completion, fallback-consume any unconsumed required facts and release/reset the director.

Milestone 8 - Karathos-style multi-hit and AOE validation

- Test one Impact0 consuming N targets simultaneously.
- Test Impact0..Impact4 consuming five sequential HitIndexes.
- Test one target dying on Impact3 while Impact4 has no target fact; optional marker should no-op safely.
- Test target presenter missing; facts still consume and battle presentation completes.

Milestone 9 - Presentation scheduler and clock

- Add PrimaryAction, Reaction, Overlay, UI, Camera lanes/claims only as needed by the vertical slice.

- Add PresentationClock speed = 1x/2x and verify Timeline + generic number tween stay synchronized.
- Implement SkipNonEssential: stop/fast-forward Timeline, consume required slots, reconcile and continue.

Milestone 10 - State projection MVP

- Define BattlePresentationState and CharacterPresentationState plus revision.
- Publish a full snapshot at battle start and basic deltas for Turn/Phase/AP/HP.
- PresentationWorld mirror drives basic UI labels and character HP view state.
- Keep visible HP transition fact-driven during queued damage, then reconcile to authoritative HP when action presentation completes.

Milestone 11 - Hand/turn/action-point state

- Add HandPresentationState and CardPresentationState keyed by runtime IDs and hand slot/index.
- Use facts for draw/play animation timing where helpful, but state as final membership truth.
- Add turn/action-point UI without reading SimulationWorld directly.

Milestone 12 - Validation, profiling, and hardening

- Content validator checks markers, role tracks, fallback recipes, sockets, and asset references.
- Profile Timeline graph creation/rebinding, Animator/VFX cost, GC allocations, damage number pooling, UI rebuilds, and bridge allocations.
- Only if Timeline backend cost is materially problematic for a real category of content, add a Program/Playables backend behind the same ExecutionPlan contract.

Part X - How Much Work Does Future Content Require?

46. New character with standard combat vocabulary

After the pipeline exists, a conventional new character should require almost no presentation-engine code. Gameplay design creates the simulation programs/behaviours and chooses a presentation key per action. Animation produces clips. Presentation assembly creates or variants the Timeline recipes for distinctive attacks and places semantic markers. VFX/audio/camera assets are attached. The validator/baker registers the content. Generic damage/heal/death/effect systems automatically work because the facts are standardized.

47. New character with an unusual multi-hit pattern

If the simulation can already represent the hit semantics, presentation work is usually just marker/slot authoring. A 17-hit FTS might use a mixture of precise markers and burst/range markers. No engine code is required simply because the hit count is larger. Engine extension is only needed if the mechanic introduces a genuinely new semantic concept that existing facts/slots cannot represent.

48. New global state/UI value

Add the value to an appropriate presentation projection and delta, then bind a PresentationWorld view model/UI component. This should not require touching the Timeline backend. If the value has an animated transition, optionally add a fact that describes the transition while the projection remains the authoritative final state.

49. New bespoke cinematic presentation type

Most bespoke actions can still use the Timeline backend with new custom tracks/markers or additional standardized binding roles. If a future category truly does not fit Timeline - for example a procedurally generated sequence with thousands of tiny scheduled operations - implement a new backend behind PresentationBackendKind. Facts, state, action packets, semantic slots, scheduler, registry, and bridge remain intact.

Part XI - Architecture Risks and Deliberate Escape Hatches

Risk	Mitigation / escape hatch
Timeline becomes too heavy for frequent tiny actions	Those actions route Generic today; add Program backend later if needed.
Timeline package/API changes	Timeline-specific code is isolated in TimelinePresentationExecutor and recipe adapter.
Need procedural sequencing not comfortable in Timeline	Program/PlayableGraph backend consumes same ExecutionPlan/slot contracts.
Content authors misuse arbitrary UnityEvents	Do not expose UnityEvent as the combat contract; use typed Archeus markers/roles.
Presentation backlog grows	PresentationClock speed-up, skip policies, look-ahead planner, generic compression, never block simulation.
State and facts diverge visually	Revised state snapshot is correctness backstop; reconcile at safe points.
Hidden simulation info leaks to visuals	Typed emitter + whitelisted payloads + no arbitrary component access.
Cross-world lifetime bugs	Stable RuntimeIDs + copy-only bridge + local world ownership.

Part XII - Decision Matrix: Why This Is the Recommended Commitment

Criterion	Raw Timeline everywhere	Custom VM everywhere	Recommended hybrid/backend-neutral core
Animator authoring UX	Excellent	Poor until substantial custom tooling exists	Excellent for authored actions
Common DOT/status overhead	Unnecessarily heavy	Excellent	Excellent via Generic backend
Dynamic target scale	Awkward if modeled as tracks	Excellent	Excellent via semantic slot fan-out
Cinematic FTS authoring	Excellent	Requires major editor/tooling investment	Excellent via Timeline
Runtime control	Moderate	Excellent	Excellent above backend + sufficient backend control
Future backend replacement	Poor if contracts depend on Timeline	Poor if all content depends on VM opcodes	Strong: plans/slots are backend-neutral
MVP implementation cost	Low initially, architectural debt later	High	Moderate and focused on game-specific value
Live-service content scaling	Moderate without wrapper	Potentially high after tooling investment	High from the start
Failure/fallback policy	Ad hoc unless custom layer added	Can be built in	Built into planner/runner contracts

FINAL ARCHITECTURAL COMMITMENT

Commit to: immutable presentation-safe facts; versioned presentation-safe state; stable RuntimeIDs; explicit ActionInstanceID/HitIndex semantics; copy-only world bridges; backend-neutral PresentationExecutionPlan; semantic slot contracts; a presentation scheduler/clock; generic fact presenters; strict fallback/reconciliation. Use Unity Timeline/Playables as the first and primary authored-action backend, not as the core semantic architecture. This is the choice that provides high-end authoring today while keeping the door open to a future custom program backend

without reworking the simulation/presentation boundary.

Appendix A - Suggested Initial Type Inventory

```
Simulation / bridge
  PresentationFact
  PresentationFactType
  PresentationFactPayload
  PresentationFactSequenceCounter
  PresentationEmissionContext
  PresentationFactEmitter
  PresentationFactBridgeExportSystem

Presentation planning
  PresentationActionPacket
  PresentationFactAssembler
  PresentationRecipeID
  ActionPresentationDefinition
  PresentationBackendKind
  PresentationExecutionPlan
  PresentationFactSlotBinding
  PresentationPresenterBinding
  PresentationPlanCompiler
  PresentationRouter

Runtime shell
  BattlePresentationScheduler
  BattlePresentationRunner
  PresentationClock
  PresentationRegistry
  PresentationAssetRegistry
  PresentationPoolRegistry

Timeline backend
  TimelinePresentationRecipe
  TimelinePresentationExecutor
  PresentationConsumeMarker
  PresentationTrackRole
  PresentationSlotID
  PresentationFactConsumptionMode

Generic backend
  GenericPresentationExecutor
  DamageFactPresenter
  HealFactPresenter
  EffectFactPresenter
  DeathFactPresenter
  DamageNumberPool
  HitReactionPresenter

State bridge
  BattlePresentationState
  CharacterPresentationState
  HandPresentationState
  PresentationStateRevision
  PresentationStateSnapshot
  PresentationStateDelta
  PresentationStateBridgeExportSystem
  PresentationStateMirrorSystem
```

Appendix B - Example Karathos Normal Attack at Runtime

```
SIMULATION (already resolved)

ActionStarted    Action=581 Definition=KarathosNormal Source=103
DamageApplied    Action=581 Hit=0 Target=201 Amount=12400
DamageApplied    Action=581 Hit=0 Target=202 Amount=17100 Crit
DamageApplied    Action=581 Hit=0 Target=203 Amount=11200
```

```

EntityKilled    Action=581 Target=203
ActionCompleted Action=581

    | copy-only fact bridge
    v

ACTION PACKET 581
    |
    | recipe lookup + slot binding
    v

EXECUTION PLAN
Backend: Timeline
Recipe: Karathos_Normal
SourceAnimator -> Presenter(103)
Impact0 -> Damage facts for 201, 202, 203
Aftermath -> EntityKilled(203)

    | scheduler grants PrimaryAction + Camera
    v

TIMELINE
0.00 play karathos_normal.anim
0.18 source trail / swing audio
0.43 Impact0 marker
    -> generic damage presenter fans out to 3 targets
0.46 camera impulse
0.58 Aftermath marker
    -> death presenter for target 203
1.10 Finish

    | completion validation
    v

All required facts consumed; director returned to pool; next plan may start.

```

Appendix C - Example State Reconciliation During the Same Attack

```

Authoritative simulation state after action resolves instantly:
Enemy 201 HP = 60,000
Enemy 202 HP = 42,900
Enemy 203 HP = 0 / dead
State Revision = 913

Presentation is still at 0.00s:
Enemy 201 visible HP = pre-hit value
Enemy 202 visible HP = pre-hit value
Enemy 203 still visually alive

At Impact0:
Damage facts animate visible HP toward authoritative values.

At Aftermath:
EntityKilled fact starts/commits death presentation for 203.

At Finish / safe reconciliation:
Presentation mirror verifies visible state against Revision 913.
Any drift is corrected.

```

Appendix D - External Technical Basis

The recommended architecture is a project design, not a copy of a published HSR implementation. The following Unity documentation supports the specific engine capabilities assumed by the Timeline/Playables backend and future asset-loading layer:

Unity 6 Playable - <https://docs.unity3d.com/cn/6000.0/ScriptReference/Playables.Playable.html>

Playables are customizable runtime objects inside PlayableGraphs and can feed outputs such as Animator-bound animation.

Unity 6 PlayableDirector - <https://docs.unity3d.com/ja/6000.0/ScriptReference/Playables.PlayableDirector.html>

PlayableDirector instantiates a PlayableGraph from a PlayableAsset and provides playback/scheduling/binding support used by Timeline.

PlayableDirector.SetGenericBinding -

<https://docs.unity3d.com/kr/2023.1/ScriptReference/Playables.PlayableDirector.SetGenericBinding.html>

Allows Timeline/Playable output bindings to be associated with runtime scene objects.

Timeline marker / custom marker APIs - <https://docs.unity3d.com/ja/Packages/com.unity.timeline%401.5/api/UnityEngine.Timeline.TrackAsset.CreateMarker.html>

Timeline supports custom marker types and notifications, which is the basis for typed Archeus semantic consumption markers.

Timeline custom marker sample - https://docs.unity3d.com/kr/Packages/com.unity.timeline%401.6/manual/smpl_custom_annotation.html

Shows the supported custom-marker authoring workflow and editor customization.

PlayableExtensions.SetSpeed (Unity 6) -

<https://docs.unity3d.com/ja/current/ScriptReference/Playables.PlayableExtensions.SetSpeed.html>

Playables expose a speed multiplier that changes local-time progression, supporting variable presentation speed.

Unity Addressables LoadAssetsAsync - <https://docs.unity3d.com/kr/Packages/com.unity.addressables%401.21/api/UnityEngine.AddressableAssets.Addressables.LoadAssetsAsync.html>

Supports asynchronous loading of assets and dependencies, useful for later battle presentation preloading/residency.

Closing note

The architectural goal is not to predict every future character mechanic today. It is to make future presentation capabilities additive instead of invasive. The fact/state boundaries, semantic action identity, backend-neutral plan, and reconciliation rules are the parts that should be treated as foundational. Timeline, generic presenters, pooling, asset loading, and even a future custom VM are implementations around those stable contracts.